



PharmaGest

Pilotée par vous, où que vous soyez

DOCUMENTATION PRODUIT — VERSION 2.4 CONSOLIDÉE

Spécifications Techniques & Fonctionnelles — SaaS Multi-Pharmacies

Champ	Valeur
Produit	PharmaGest — Logiciel de gestion de pharmacie
Version	v2.4 — Document consolidé (v2.0 + v2.1 + v2.2 + v2.3 + v2.4)
Date	Avril 2026 — Dernière mise à jour : 17/04/2026
Marché cible	Afrique francophone — Guinée en priorité
Modèle commercial	Licence unique (500k–800k GNF) + maintenance annuelle optionnelle
Statut	Phase 1 ☒ · Migration v2.2 ☒ · Migration v2.3 ☒ · Session 8 ☒ · Phase 2 en cours
Stack	Next.js 14 · PostgreSQL · Prisma 5.22.0 · NextAuth · Vercel/Supabase

1. Vision & Positionnement Produit

1.1 Problème résolu

En Afrique francophone, la grande majorité des pharmacies indépendantes sont gérées sans outil numérique adapté. Les propriétaires utilisent des cahiers, des tableurs Excel non structurés, ou des logiciels génériques inadaptés au contexte local. Résultat : stock mal maîtrisé, pertes sur péremptions, caisse non contrôlée, et impossibilité pour le propriétaire de surveiller son activité à distance.

PharmaGest résout ce problème en offrant une plateforme web complète, accessible depuis n'importe quel navigateur, conçue spécifiquement pour les pharmacies africaines francophones.

1.2 Proposition de valeur

Pour qui	Ce que PharmaGest apporte
Propriétaire absent	Tableau de bord temps réel accessible depuis son téléphone ou PC, depuis n'importe où
Pharmacien gestionnaire	Contrôle total du stock, des lots, des péremptions et des commandes fournisseurs
Caissier / Employé	Interface de vente (POS) rapide, intuitive et sans formation longue
Direction (multi-pharmacies)	Vue consolidée sur toutes les pharmacies du groupe depuis une seule session

1.3 Marché cible

Marché primaire : pharmacies indépendantes en Afrique francophone (Guinée, Sénégal, Côte d'Ivoire, Mali, Cameroun, RDC...). Taille estimée : plusieurs dizaines de milliers de pharmacies indépendantes dans la zone.

Marché secondaire : groupements de pharmacies, grossistes pharmaceutiques, parapharmacies.

1.4 Modèle commercial

Élément	Détail
Type de licence	Licence unique par pharmacie (achat one-time)
Mise à jour majeure (v3.0+)	Facturée séparément — optionnel
Maintenance annuelle	Contrat optionnel incluant support prioritaire et mises à jour mineures
Installation & formation	Prestation facturée à part (déplacement ou visioconférence)
Essai	30 jours d'essai gratuit sans carte bancaire

Pourquoi la licence unique et pas l'abonnement mensuel ? La licence unique est le modèle le plus accepté en Afrique de l'Ouest. Elle évite la résistance culturelle à l'abonnement et permet au développeur de livrer un produit stable sans contrainte de support continu.

1B. Étude Concurrentielle & Positionnement

Concurrent	Profil	Présence	Faiblesse vs PharmaGest
AS PHARM	Leader béninois 2016. SaaS abonnement. Stock, ventes, facturation.	Bénin, Sénégal, Niger	Guinée non couverte — abonnement = résistance culturelle
FlexPharm	Solution malienne. SaaS CFA : Mobile Money, TVA locale, lots.	Mali — zone XOF	Pas de support GNF natif
Tanél Health	Startup sénégalaise 2020. Interface moderne, stocks, ventes.	Sénégal — limité	Peu de visibilité en Guinée
Winpharma	Solution française 30 ans. Très complète.	France principalement	Inadaptée marché africain

Différenciateur	Explication & Avantage concurrentiel
1. Propriétaire absent — priorité n°1	Dashboard temps réel, inventaire à distance, alertes immédiates. Cœur du slogan et de toute l'UX.
2. Fait en Guinée, pour la Guinée	Devise GNF native, pharmacie pilote locale, connaissance du terrain. AS PHARM ne couvre pas ce marché.
3. Licence unique — "tu paies une fois"	Modèle le plus accepté en Afrique de l'Ouest. Élimine la résistance à l'abonnement.

1C. Budget & Pricing

Service	Phase pilote (gratuit)	Production commerciale
Vercel	Plan Hobby — Gratuit	Plan Pro — 20 USD/mois
Supabase	Plan gratuit — 500 Mo	Plan Pro — 25 USD/mois
Backblaze B2	Plan gratuit — 10 Go	~1 USD/mois
Domaine	~15 USD/an	~15 USD/an
TOTAL	≈ 15 USD/an	~46 USD/mois ≈ 40 000 GNF/mois

Formule	Prix & Détail
Licence unique	500 000 — 800 000 GNF selon taille pharmacie — accès permanent
Installation & formation	200 000 GNF — présentiel ou visio
Maintenance annuelle (optionnel)	150 000 GNF/an — mises à jour mineures + support email
Mise à jour majeure v3.0+	250 000 GNF — facturée séparément, optionnelle
Essai gratuit	30 jours — accès complet sans engagement

2. Architecture Technique

2.1 Stack technologique — versions forcées (non rediscutables)

⚠ Décision de rupture : on abandonne Electron + SQLite local au profit d'une architecture web full-stack. Raison : accès distant en temps réel, multi-utilisateurs, déploiement simplifié, évolutivité.

Couche	Technologie	Version	Justification
Framework	Next.js (App Router)	14.x	React SSR/CSR, routing intégré, déploiement Vercel trivial
Langage	TypeScript	5.x	Typage fort, maintenabilité long terme
UI	shadcn/ui + Tailwind CSS	latest	Composants professionnels prêts à l'emploi
Base de données	PostgreSQL	15.x	Robuste, open-source, excellente gestion multi-tenant
ORM	Prisma	5.22.0 (forcé)	npm installe Prisma 7 par défaut — incompatible. Forcé à 5.22.0
Authentification	NextAuth.js	4.x	Auth complète : sessions, JWT, gestion des rôles
Temps réel	Server-Sent Events	natif Next.js	Mises à jour temps réel sans WebSocket complexe
Hébergement app	Vercel	Plan gratuit	Déploiement automatique, CDN mondial, SSL inclus
Hébergement BDD	Supabase	Plan gratuit	PostgreSQL managé, 500 Mo, stable
Backup BDD	Backblaze B2	Plan gratuit	Gratuit jusqu'à 10 Go — dumps compressés
Graphiques	Recharts	latest	Bibliothèque React légère pour dashboards
PDF	@react-pdf/renderer	latest	Génération reçus et rapports PDF
Validation	react-hook-form + Zod	7.x + 3.23.8 (forcé)	Zod 4.x incompatible avec @hookform/resolvers 3.9.0
@hookform/resolvers	—	3.9.0 (forcé)	Version 5.x incompatible avec react-hook-form 7.x

2.2 Architecture Multi-tenant

Une seule instance de l'application sert toutes les pharmacies clientes. L'isolation des données est garantie par un champ `pharmacieId` présent sur toutes les tables.

```
//  Pattern obligatoire — toujours findFirst avec filtre pharmacieId
const item = await prisma.medicament.findFirst({
  where: { id: params.id, pharmacieId: session.user.pharmacieId }
});
if (!item) return notFound();
```

```
// ✗ Jamais sans le filtre — fuite de données inter-pharmacies
const item = await prisma.medicament.findUnique({ where: { id: params.id } });
```

2.3 Rôles utilisateurs

Rôle	Description
super_admin	Compte PharmaGest (développeur). Accès à toutes les pharmacies. Gestion des licences.
admin	Propriétaire ou gérant. Accès complet à SA pharmacie.
caissier	Employé. Accès restreint : ventes, consultation stock, session caisse.

2.4 Contraintes réseau — Guinée

[⚠ Contrainte permanente : les ports 5432 et 6543 sont bloqués par le réseau en Guinée. Toutes les migrations sont appliquées manuellement via le SQL Editor de Supabase.](#)

Décision	Détail
Migration SQL	Appliquée via SQL Editor Supabase — jamais via prisma migrate dev
Deux fichiers .env	.env (Prisma/seed) · .env.local (Next.js) — port 6543 pooler pgbouncer
NEXTAUTH_SECRET	node -e "require('crypto').randomBytes(32).toString('base64')" — openssl non disponible Windows
bcryptjs	bcryptjs (pas bcrypt) — bcrypt nécessite binaires natifs incompatibles Windows/Vercel
Police	Inter (Google Fonts) — Geist non disponible dans Next.js 14
findFirst	Toujours findFirst avec pharmacieId — jamais findUnique sans filtre tenant
Prisma.JsonNull	Pour les champs Json nullable — null TypeScript n'est pas assignable
Controller (RHF)	Utiliser Controller (pas register) avec shadcn/ui Input
useSearchParams	Toujours isolé dans composant enfant + <Suspense> — exigence Next.js 14 production
cn() dans utils.ts	Ne jamais écraser lib/utils.ts — shadcn/ui l'importe pour la fusion de classes Tailwind

2.5 Champs Prisma vérifiés — pièges connus ⚡ v2.4

[⚠ Toujours vérifier les champs réels dans schema.prisma avant d'écrire une requête. Ne jamais supposer qu'un champ existe parce qu'il semble logique.](#)

Modèle	Champ supposé	Réalité
CommandeFournisseur	montantTotal	✗ N'existe pas — calculer depuis items si besoin

Modèle	Champ supposé	Réalité
SessionCaisse	statut	✗ N'existe pas — session ouverte = dateCloture: null + actif: true

```
// ✔ Session caisse ouverte — pattern correct
await prisma.sessionCaisse.findFirst({
  where: { pharmacieId, userId, dateCloture: null, actif: true }
})
```

3. Modèle de Données

Schéma complet dans prisma/schema.prisma — 18 tables en BDD au 17/04/2026.
pharmacield est présent sur toutes les tables métier.

Règles clés :

- médicaments.prix_achat — n'existe pas (varie par lot — dans lots uniquement) ⚡ v2.1
- stock_actuel — calculé dynamiquement : SUM(quantite_actuelle) FROM lots WHERE actif = true
- commandes_fournisseur.montantTotal — n'existe pas ⚡ v2.4
- sessions_caisse.statut — n'existe pas — session ouverte = dateCloture: null + actif: true ⚡ v2.4
- FIFO : lot avec datePeremption la plus proche décrémenté en priorité
- audit_logs : écriture seule — champs entite et entiteld (pas tableCible / recordId)
- Utiliser Prisma.JsonNull pour les champs Json nullable
- mouvements_stock.userId : nullable en BDD (historique) — OBLIGATOIRE dans tout nouveau code ⚡ v2.3

3.0 Vue d'ensemble — 18 tables

Table	Rôle	Version
pharmacies	Table racine multi-tenant	v2.0
users	Utilisateurs liés à une pharmacie	v2.0
medicaments	Produits — sans prix_achat (dans lots)	v2.1
categories	Référentiel catégories médicaments	v2.0
formes_galeniques	Référentiel formes galéniques	v2.0
lots	Lots — prixAchat NOT NULL, FIFO	v2.1
fournisseurs	Fournisseurs de la pharmacie	v2.0
commandes_fournisseur	Commandes avec workflow de statut	v2.0
commande_items	Lignes de commande	v2.0
ventes	Ventes liées à une session_caisse_id	v2.0
vente_items	Lignes de vente — lot_id FK (FIFO), prix snapshot	v2.0
mouvements_stock	Journal de tous les mouvements — +userId	v2.3
sessions_caisse	Sessions caisse — ouverture/clôture/écart	v2.0
audit_logs	Journal immuable des actions sensibles	v2.0
inventaires	Sessions d'inventaire hebdomadaire	v2.2
lignes_inventaire	Lignes de comptage par médicament	v2.2
categories_depenses	Types de dépenses configurables	v2.3
depenses	Chaque dépense enregistrée	v2.3

3.1 à 3.11 — Détail des tables

(Les détails champ par champ de chaque table sont identiques à la version v2.3. Seules les mises à jour v2.4 sont documentées ci-dessous.)

3.8 Table : sessions_caisse — Correction v2.4

Le champ statut n'existe PAS sur cette table. Une session est considérée ouverte quand dateCloture IS NULL et actif = true.

Champ	Type	Description
id	TEXT UUID PK	Identifiant de la session
pharmaciId / userId	TEXT FK	Tenant et caissier responsable
montantOuverture	DECIMAL NOT NULL	Fonds déclaré à l'ouverture
montantCloture / montantAttendu	DECIMAL	Montants clôture (null si en cours)
ecart	DECIMAL	montantCloture – montantAttendu
dateOuverture	DATETIME NOT NULL	Horodatage d'ouverture
dateCloture	DATETIME	null si session en cours — clé pour détecter session ouverte
noteCloture	TEXT	Obligatoire si ecart ≠ 0
actif	BOOLEAN DEFAULT true	false si clôturée — clé pour détecter session ouverte
createdAt / updatedAt	DATETIME	Horodatages automatiques

3.12 Codes audit_logs standardisés

PRICE_UPDATE · STOCK_ADJUST · SALE_CANCEL · USER_LOGIN · USER_LOGOUT · USER_CREATE · ROLE_CHANGE · STOCK_MIN_UPDATE · BACKUP_CREATED · SESSION_OPEN · SESSION_CLOSE · PRODUCT_ARCHIVE · INVENTAIRE_CREE · INVENTAIRE_VALIDE · INVENTAIRE_ECART · DEPENSE_AJOUTEE ⚡ v2.3

4. Fonctionnalités

*RÈGLE D'OR : Ne jamais passer à une fonctionnalité **BONUS** tant que toutes les fonctionnalités **CORE** ne sont pas stables et testées.*

Module	Fonctionnalité	Priorité	Description
 Auth	Connexion sécurisée	CORE	Email + mot de passe, session NextAuth, redirection selon rôle
 Auth	Période d'essai 30j	CORE	Accès complet, bannière de rappel à J-7, blocage si expirée
 Dashboard	KPIs temps réel	CORE	CA jour, nb ventes, valeur stock, alertes actives — SSE
 Dashboard	Alertes visuelles	CORE	Stock bas, péremptions < 30j, seuils dépenses dépassés
 Dashboard	Graphique ventes	CORE	Courbe CA des 7 derniers jours (Recharts)
 Dashboard	Actions rapides ⚡v2.4	BONUS	Boutons Nouvelle vente, Ouvrir caisse, Lancer inventaire directement depuis le dashboard
 Médicaments	CRUD complet	CORE	Liste paginée, ajout, modification, archivage logique, fiche détail
 Stock	Vue d'ensemble	CORE	Valeur totale, alertes, produits critiques
 Stock	Gestion des lots	CORE	Liste lots par médicament, dates péremption, quantités
 Stock	Historique mouvements	CORE	Journal filtrable par produit, lot, date, type
 Stock	Alertes péremption	CORE	Lots expirant dans 30/60/90 jours
 Inventaire ⚡v2.2	Lancer inventaire	CORE	Admin lance une session de comptage pour tous les médicaments actifs
 Inventaire ⚡v2.2	Tableau de saisie	CORE	Grille : nom, stock système, stock réel saisi, écart calculé
 Inventaire ⚡v2.2	Validation + ajustement	CORE	Admin valide → ajustement automatique lots via transaction Prisma
 Inventaire ⚡v2.2	Historique inventaires	CORE	Liste des inventaires passés avec statuts
 Dépenses ⚡v2.3	Catégories configurables	CORE	Admin configure les types de dépenses avec seuil d'alerte mensuel
 Dépenses ⚡v2.3	Saisie dépense	CORE	Admin et Caissier saisissent une dépense avec catégorie et montant
 Dépenses ⚡v2.3	Total mensuel + alertes	CORE	Total par catégorie + indicateur si seuilAlerte dépassé
 Dépenses ⚡v2.3	Archivage logique	CORE	Admin archive — jamais de suppression physique

Module	Fonctionnalité	Priorité	Description
 Ventes	Interface POS	CORE	Recherche produit, panier, calcul total, monnaie rendue
 Ventes	Décrément FIFO	CORE	Lot le plus proche péremption décrémenté automatiquement
 Ventes	Impression reçu PDF	CORE	Reçu généré et imprimable depuis le navigateur
 Ventes	Annulation vente	CORE	Admin uniquement, motif obligatoire, remise en stock
 Ventes	Sessions caisse	CORE	Ouverture, clôture, calcul écart, rapport clôture
 Ventes	Paieement partiel ⚡v2.4	CORE	Client paie une partie, motif du reste (crédit, prise en charge)
 Ventes	Modes de paiement ⚡v2.4	CORE	Cash, mobile money (Orange/MTN), paiement marchand, mixte
 Ventes	Remises ⚡v2.4	CORE	Remise en % ou montant fixe appliquée sur la vente
 Ventes	Partage WhatsApp ⚡v2.4	BONUS	Partager la facture ou commande sur WhatsApp via lien wa.me
 Clients ⚡v2.4	Fiche client	CORE	Nom, téléphone, historique achats, solde crédit en cours
 Clients ⚡v2.4	Crédit client	CORE	Plafond par client, solde calculé, blocage si plafond atteint
 Clients ⚡v2.4	Relances automatiques	CORE	WhatsApp/SMS à J-3, Jour J, J+7 après échéance
 Fournisseurs	CRUD + workflow commande	CORE	Brouillon → Envoyée → Reçue → création lots auto
 Fournisseurs	Commandes suggérées ⚡v2.4	BONUS	Si stock < stockMinimum, proposition auto de commande pré-remplie
 Rapports	Rapport ventes	CORE	Journalier, période, par caissier, par mode paiement
 Rapports	Rapport inventaire ⚡v2.2	CORE	Écarts par médicament + valeur totale des pertes
 Rapports	Rapport bénéfice net ⚡v2.3	CORE	CA – CMV – Dépenses + ligne trésorerie séparée
 Rapports	Rapport crédits clients ⚡v2.4	CORE	Soldes dus par client, échéances, retards
 Rapports	Journal d'activité	CORE	audit_logs filtrable — Admin uniquement
 Notifs	Email alertes stock	CORE	Email automatique quand stock < stockMinimum
 Notifs	Email seuil dépenses ⚡v2.3	CORE	Email si total mensuel catégorie dépasse seuilAlerte

Module	Fonctionnalité	Priorité	Description
 Notifs	Relances crédit ⚡v2.4	CORE	WhatsApp/SMS automatiques selon échéance de crédit
 Personnel	Gestion employés	CORE	Liste, créer, modifier rôle, désactiver — tracé audit
 Super Admin	Liste + activation licences	CORE	Toutes les pharmacies clientes, activation/désactivation licence

5. Pages & Navigation

5.1 Routes — Espace Pharmacie

Route	Accès
/dashboard	Admin + Caissier — KPIs temps réel, alertes, graphique CA
/ventes/nouvelle	Admin + Caissier — Interface POS
/ventes	Admin + Caissier — Historique des ventes
/stock	Admin + Caissier — Vue stock et alertes
/stock/inventaire	Admin — Liste inventaires + Lancer ⚡ v2.2
/stock/inventaire/[id]	Admin + Caissier — Tableau de saisie ⚡ v2.2
/medicaments	Admin (écriture) / Caissier (lecture)
/caisse/session	Admin + Caissier — Session caisse
/clients	Admin + Caissier — Liste et fiches clients ⚡ v2.4
/clients/[id]	Admin + Caissier — Fiche client + historique + crédits ⚡ v2.4
/credits	Admin uniquement — Tableau de bord crédits en cours ⚡ v2.4
/depenses	Admin + Caissier — Saisie et liste dépenses ⚡ v2.3
/depenses/categories	Admin uniquement — Gérer les catégories ⚡ v2.3
/fournisseurs	Admin uniquement
/fournisseurs/commandes	Admin uniquement
/rapports	Admin uniquement
/rapports/audit	Admin uniquement
/personnel · /parametres	Admin uniquement

5.2 Espace Super Admin

Route	Description
/superadmin	Dashboard global — nb pharmacies, licences actives
/superadmin/pharmacies	Liste de toutes les pharmacies clientes
/superadmin/pharmacies/[id]	Fiche pharmacie — activation/désactivation licence

6. Règles Métier

6.1 Règles de stock & lots (FIFO)

- Chaque réception crée obligatoirement un lot avec numéro de lot et date de péremption.
- Vente : système sélectionne automatiquement le lot avec la date de péremption la plus proche (FIFO). L'utilisateur ne choisit pas le lot manuellement.
- Impossible de vendre si stock = 0. Blocage avec message explicite. Zéro tolérance.
- Ajustement manuel : motif obligatoire (non-nullable). Tracé dans mouvements_stock avec quantiteAvant et quantiteAprès dans la même transaction.
- Lot dont quantiteActuelle atteint 0 → automatiquement marqué actif = false.
- userId dans mouvements_stock : nullable en BDD (compat. historique) — OBLIGATOIRE dans tout nouveau code Phase 2+. ⚡ v2.3

6.2 Règles de vente

- Création d'une vente = transaction atomique. Si une opération échoue → ROLLBACK Prisma. Pas de vente partielle.
- prixUnitaire dans vente_items = prix au moment de la vente — immuable.
- Monnaie rendue ne peut jamais être négative. Bloqué si montantPayé < montantTotal (sauf paiement partiel avec motif).
- Annulation (Admin uniquement, motif obligatoire) → remet les quantités dans les lots d'origine + mouvement retour.
- Statut "annulee" : stock non affecté. Statut "remboursee" : stock réintégré via TypeMouvement.retour. ⚡ v2.3
- CA net = SUM(montantTotal) WHERE statut = 'completee' uniquement — annulee et remboursee exclus. ⚡ v2.3
- Paiement partiel : montantPayé < montantTotal autorisé si motif renseigné. Crédit enregistré dans credit_vente. ⚡ v2.4
- Remise : montant ou pourcentage appliqué avant calcul du total, enregistré dans vente_items. ⚡ v2.4

6.3 Règles de session caisse

- Impossible d'effectuer une vente sans session caisse ouverte.
- Une seule session ouverte à la fois par utilisateur.
- Si ecart ≠ 0 à la clôture → noteCloture obligatoire.
- Plusieurs caisses simultanées : chaque caissier a sa propre session (userId différent) — pas de gêne mutuelle.

6.4 Règles de bénéfice net ⚡ v2.3

```
CA net      = SUM(montantTotal) WHERE statut = 'completee'
CMV         = SUM(vente_items.quantite × lots.prixAchat) - ventes completee
uniquement
Dépenses    = SUM(montant) FROM depenses WHERE archivee = false
```

```
Bénéfice net = CA net - CMV - Dépenses
```

```
Info trésorerie (rapport Session 22 - ligne séparée) :
```

```
Achats reçus = SUM(prixAchat × quantiteInitiale) FROM lots créés sur la période
```

Le CMV via FIFO est utilisé (pas les achats bruts) car un gros achat en janvier ne doit pas afficher une perte fictive ce mois-là — seul le coût de ce qui a réellement été vendu est déduit.

6.5 Règles du module Inventaire ⚡ v2.2

- Blocage vente zéro tolérance : si stock = 0 → vente bloquée, message explicite. Impossible de vendre en négatif.
- Admin lance → statut en_cours → employé saisit stockReel → ecart = stockReel - stockTheorique calculé auto.
- motifEcart obligatoire si ecart ≠ 0 (Zod enforce). L'inventaire ne peut pas passer en en_attente_validation tant qu'il reste des écarts sans motif.
- Seul l'Admin peut valider. Validation = transaction Prisma qui ajuste tous les lots + MouvementStock type ajustement + AuditLog INVENTAIRE_VALIDE.

6.6 Règles du module Dépenses ⚡ v2.3

- Traçabilité obligatoire : toute dépense enregistre userId (non nullable dans dépenses).
- Alerte seuilAlerte : total mensuel catégorie > seuilAlerte → alerte dashboard + email cron automatique.
- Pas de suppression physique : archivage logique uniquement via archivee = true — Admin uniquement.
- AuditLog DEPENSE_AJOUTEE généré à chaque création de dépense.
- Catégories pré-remplies dans le seed : Salaires · Loyer · Électricité & eau · Impôts & taxes · Fournitures & matériel · Réparations & entretien · Autres charges.

6.7 Règles du module Client ⚡ v2.4

- Création d'un client : nom + téléphone obligatoires. Email et institution optionnels.
- Plafond crédit : défini par l'Admin uniquement. Le caissier ne peut pas le modifier.
- Vérification plafond : avant d'accorder un crédit, soldeActuel + montantNouveauCredit ≤ plafondCredit. Si dépassé → blocage avec message explicite.
- Vente anonyme : clientId nullable dans credit_vente. Crédit sans client enregistré = pas de relance possible.
- Relances automatiques : J-3, Jour J, J+7 après échéance via WhatsApp (lien wa.me) ou email.
- Remboursement : enregistrement du montant + mode de paiement + date. Solde mis à jour automatiquement.

6.8 Règles de paiement partiel & prises en charge ⚡ v2.4

Le système unifie crédits clients et prises en charge institutionnelles en un seul concept : paiement partiel avec motif.

- Tout reste dû après une vente est enregistré dans credit_vente avec un champ motif libre.
- Motif exemples : "Prise en charge Ministère Santé" · "Mutuelle employeur XYZ" · "Crédit personnel" · "Bon hospitalier n°1234"
- Institution = client de type institution. Même table clients, même logique de plafond et de relance.

- CA net = seule la partie encaissée (montantPaye) entre dans le CA du jour. Le reste entre en CA quand il est encaissé.

6.9 Règles de scalabilité & performance ⚡ v2.4

- Index de performance créés sur toutes les colonnes FK et colonnes de filtrage fréquent.
- La boucle 7 requêtes du dashboard (CA par jour) sera remplacée par une requête GROUP BY DATE en Session 9 ou 10.
- Le nombre de pharmacies (tenants) n'impacte pas les performances si pharmacieId est toujours indexé et filtré.
- Timeout cache : vider .next après tout crash serveur — le cache webpack se corrompt.

6.10 Règles de sécurité multi-tenant

- Toute requête API vérifie d'abord la session NextAuth. Si non authentifié → 401.
- Toute requête filtre obligatoirement par pharmacieId de la session. Utiliser findFirst avec filtre — jamais findUnique sans filtre.
- Le Super Admin est le seul à pouvoir requêter sans filtre pharmacieId — routes /superadmin/*.
- Mots de passe hashés avec bcryptjs (saltRounds=12). Jamais stockés en clair.

6.11 Règles de sécurité URLs ⚡ v2.1

```
// ✓ Pattern obligatoire dans chaque page [id]
const item = await prisma.medicament.findFirst({
  where: { id: params.id, pharmacieId: session.user.pharmacieId }
});
if (!item) return notFound();
// ✗ Jamais sans le filtre tenant
const item = await prisma.medicament.findUnique({ where: { id: params.id } });
```

6.12 Règles d'audit

- Table audit_logs en écriture seule. Aucune API de modification ou suppression n'existe.
- Traçage effectué dans la même transaction que l'action. Si la transaction échoue, le log n'est pas écrit.
- Champs réels : entite et entiteId (pas tableCible / recordId).
- Utiliser Prisma.JsonNull (pas null TypeScript) pour les champs Json nullable.

6.13 Matrice des droits — complète v2.4

Action	Admin	Caissier
Se connecter	✓	✓
Voir le dashboard temps réel	✓	✓
Effectuer une vente (POS)	✓	✓
Ouvrir / clôturer sa session caisse	✓	✓
Consulter l'historique des ventes	✓	✓ (les siennes)

Action	Admin	Caissier
Voir la liste des médicaments	✓	✓ (lecture)
Annuler une vente	✓	✗
Créer / modifier un médicament	✓	✗
Gérer les lots	✓	✗
Ajuster le stock manuellement	✓	✗
Lancer un inventaire ⚡ v2.2	✓	✗
Saisir les quantités d'inventaire ⚡ v2.2	✓	✓
Valider un inventaire ⚡ v2.2	✓	✗
Voir l'historique inventaires ⚡ v2.2	✓	✗
Enregistrer une dépense ⚡ v2.3	✓	✓
Modifier / archiver une dépense ⚡ v2.3	✓	✗
Gérer les catégories de dépenses ⚡ v2.3	✓	✗
Voir les rapports dépenses ⚡ v2.3	✓	✗
Créer un client ⚡ v2.4	✓	✓
Modifier un client ⚡ v2.4	✓	✗
Définir le plafond crédit ⚡ v2.4	✓	✗
Accorder un crédit ⚡ v2.4	✓	✓ (dans la limite du plafond)
Enregistrer un remboursement ⚡ v2.4	✓	✓
Voir tous les crédits en cours ⚡ v2.4	✓	✗
Envoyer les relances ⚡ v2.4	✓	✗
Gérer les fournisseurs et commandes	✓	✗
Voir toutes les sessions caisse	✓	✗
Consulter les rapports	✓	✗
Voir le journal d'activité (audit)	✓	✗
Gérer les employés	✓	✗
Configurer les paramètres	✓	✗
Activer / désactiver des licences	Super Admin	Super Admin

7. Design System & Identité Visuelle

7.1 Logotype

Élément	Spécification
Nom du produit	PharmaGest (P majuscule, G majuscule, reste minuscule)
Icône	Initiales PG — P en blanc, G en vert #2ECC8A, sur fond bleu nuit #0D2D5E
Slogan officiel	"Pilotée par vous, où que vous soyez"

7.2 Palette de couleurs

Nom	Code Hex	Usage
Bleu marine principal	#0D2847	Sidebar fond, titres h1, texte principal
Bleu marine foncé	#0A1F3D	Sidebar fond bas, dégradé
Bleu moyen	#1A4F9F	Titres h2, liens actifs, boutons
Vert accent	#2ECC8A	Logo G, boutons primaires, succès
Vert foncé	#1aad72	Hover boutons, dégradé vert
Fond page dashboard	#EEF1F6	Fond général de l'application
Fond cartes	#FFFFFF	Cartes et panneaux
Bordures	#E4E8F0	Séparateurs, bordures cartes
Texte principal	#0D2847	Corps de texte sur fond clair
Texte secondaire	#8A93A6	Labels, textes muted
Danger	#C0392B	Erreurs, alertes critiques
Avertissement	#E67E22	Alertes modérées, péremption proche

7.3 Dégradés utilisés dans le code

Élément	Valeur CSS
Sidebar fond	linear-gradient(175deg, #0D2847, #0A1F3D, #0B2640)
Logo sidebar	linear-gradient(135deg, #2ECC8A, #1aad72)
Lien actif sidebar	linear-gradient(90deg, rgba(46,204,138,.22), transparent)
Avatar utilisateur	linear-gradient(135deg, #1a6bbf, #2ECC8A)
KPI vert	linear-gradient(135deg, #1aad72, #2ECC8A)
KPI bleu	linear-gradient(135deg, #1a5fa8, #3B82F6)
KPI orange	linear-gradient(135deg, #b45309, #F59E0B)
KPI rouge	linear-gradient(135deg, #991b1b, #EF4444)
Bouton connexion	linear-gradient(135deg, #0D2847, #1a5fa8)
Fond gauche login	linear-gradient(155deg, #0D2847, #0A1F3D, #0d3a5c)

7.4 Sidebar — comportement

Propriété	Valeur
Toggle	Clic sur le bloc logo + nom pharmacie + pilule — pas de bouton séparé
Mode étendu	230px — logo + nom pharmacie + sections + labels + barre active
Mode compact	64px — icônes seules, tooltip natif via attribut title HTML
Mobile	position:fixed, overlay sombre, hamburger dans le Header

7.5 Utilitaires — lib/utils.ts

⚠ Ne jamais écraser lib/utils.ts — shadcn/ui importe cn() depuis ce fichier. Écraser ce fichier casse tous les composants UI.

Fonction	Sortie exemple
cn(...inputs)	Fusion classes Tailwind — requis par shadcn/ui — NE PAS SUPPRIMER
formatDate(date)	"2 avril 2026"
formatDateCourt(date)	"02/04/2026"
formatMontant(15000)	"15 000 GNF"
generateNumero("VTE")	"VTE-20260402-4F2A"

7.6 Utilitaires — lib/audit.ts (createAuditLog)

Paramètre	Type · Obligatoire
pharmaciId	string · ☒ obligatoire
userId	string · non obligatoire
action	string · ☒ — ex: "DEPENSE_AJOUTEE", "STOCK_ADJUST"
entite	string · non — ex: "Depense", "Medicament"
entiteId	string · non — UUID de l'enregistrement
ancienneValeur / nouvelleValeur	Record<string, unknown> · non
ipAddress	string · non

8. Roadmap de Développement

RÈGLE D'OR : Une fonctionnalité = un test immédiat = un commit. Ne jamais accumuler des fonctionnalités non testées.

Phase 1 ☒ TERMINÉE — 02/04/2026 · 7 sessions · Zéro dette technique

Migration v2.2 ☒ TERMINÉE — 06/04/2026 (16 tables)

Migration v2.3 ☒ TERMINÉE — 11/04/2026 (18 tables)

Session 8 ☒ TERMINÉE — 17/04/2026 (Dashboard données réelles)

Phase 2 — MVP Core (Sessions 8 à 16)

Session	Contenu
Session 8 ☒	Dashboard avec vraies données Prisma — CA réel, alertes stock, péremptions
Session 9	Module Médicaments — Routes API CRUD
Session 10	Module Médicaments — Interface (liste, ajout, fiche)
Session 11	Module Lots + Fonction getLotFifo (FIFO)
Session 12	Module Fournisseurs
Session 12B ⚡ v2.2	Module Inventaire complet (API + interface + validation + email)
Session 12C ⚡ v2.3	Module Dépenses — API + interface (saisie + liste + alertes seuilAlerte)
Session 13	Module Stock — Vue d'ensemble + Journal des mouvements
Session 14	Sessions Caisse (ouverture/clôture/écart)
Session 14B ⚡ v2.4	Module Client — Fiches, crédits, plafonds, relances
Session 15	Interface POS avec blocage zéro tolérance + paiement partiel + remises + modes de paiement
Session 16	Impression reçu PDF + Historique ventes + Annulation

Phases 3 à 5

Phase	Objectif
Phase 3 (Sessions 17-21)	Dashboard SSE temps réel + alertes dépenses + emails + Commandes fournisseurs + Journal audit
Phase 4 (Sessions 22-25)	Rapports (ventes, stock, inventaire, bénéfice net, crédits clients) + Personnel + Paramètres + Super Admin
Phase 5 (Sessions 26-27)	Export PDF/Excel + Tests sécurité + PWA offline + Mise en production pharmagest.app 🚀

Tableau récapitulatif — 28 sessions

N°	Phase	Session	Durée	Statut
1	Phase 1	Initialisation projet Next.js	1h30-2h	✓ Fait
2	Phase 1	Schéma Prisma — toutes les tables	2h	✓ Fait
3	Phase 1	Client Prisma + Seed de données	1h30	✓ Fait
4	Phase 1	NextAuth — Authentification	2h	✓ Fait
5	Phase 1	Middleware + Protection des routes	1h30	✓ Fait
6	Phase 1	Layout principal + Sidebar	2h	✓ Fait
7	Phase 1	Audit + Utilitaires	1h	✓ Fait
8	Phase 2	Dashboard données réelles	2h	✓ Fait
9	Phase 2	Module Médicaments — API	2h	□ À faire
10	Phase 2	Module Médicaments — Interface	2h	□ À faire
11	Phase 2	Module Lots + FIFO	1h30	□ À faire
12	Phase 2	Module Fournisseurs	1h30	□ À faire
12B ⚡	Phase 2	Module Inventaire — v2.2	2h	□ À faire
12C ⚡	Phase 2	Module Dépenses — v2.3	2h	□ À faire
13	Phase 2	Module Stock — Vue & Mouvements	2h	□ À faire
14	Phase 2	Sessions Caisse	1h30	□ À faire
14B ⚡	Phase 2	Module Client — v2.4	2h	□ À faire
15	Phase 2	Interface POS (zéro tolérance + paiement partiel)	2h	□ À faire
16	Phase 2	Reçu PDF + Historique ventes	1h30	□ À faire
17	Phase 3	Dashboard SSE temps réel	2h	□ À faire
18	Phase 3	Graphiques + Alertes enrichies	1h30	□ À faire
19	Phase 3	Emails automatiques + Backup BDD	2h	□ À faire
20	Phase 3	Module Commandes Fournisseurs	2h	□ À faire
21	Phase 3	Journal Audit	1h	□ À faire
22	Phase 4	Rapports (bénéfice net + inventaire + crédits)	2h	□ À faire
23	Phase 4	Personnel + Paramètres	1h30	□ À faire
24	Phase 4	Super Admin + Licences	1h30	□ À faire
25	Phase 4	PWA Offline (Service Worker + IndexedDB)	2h	□ À faire
26	Phase 5	Export rapports + Tests sécurité	2h	□ À faire

N°	Phase	Session	Durée	Statut
27	Phase 5	Optimisations + Domaine + Lancement 	2h	☐ À faire

9. Conventions de Développement

9.1 Nommage

Élément	Convention & Exemple
Composants React	PascalCase → MedicamentCard.tsx, VenteModal.tsx
Fichiers pages Next.js	kebab-case → /medicaments/[id]/page.tsx
Variables & fonctions	camelCase → stockActuel, validerVente()
Constantes globales	SCREAMING_SNAKE_CASE → STOCK_MIN_DEFAULT
Types & interfaces TypeScript	PascalCase → Medicament, LigneVente, UserRole
Tables base de données	snake_case pluriel → medicaments, vente_items
Routes API	kebab-case → /api/ventes/[id]/annuler
Commits Git	feat(stock): ajouter décrétement FIFO automatique des lots

9.2 Règles de code TypeScript

- Interdiction du type any — tous les types définis dans /types/*.ts
- Toutes les routes API retournent : { success: boolean, data?: T, error?: string }
- Toute opération multi-table utilise une transaction Prisma : prisma.\$transaction([...])
- Formulaires : react-hook-form (Controller, pas register) + Zod côté client ET serveur
- Interdiction d'appeler Prisma directement dans un composant React — toujours via une route API
- useSearchParams toujours isolé dans un composant enfant + <Suspense> — exigence Next.js 14 production Vercel

9.3 Sécurité — règles non-négociables

- Chaque route API commence par : const session = await getSession(); if (!session) return 401
- Chaque requête Prisma inclut : where: { pharmacieId: session.user.pharmacieId }
- Utiliser findFirst avec filtre pharmacieId — jamais findUnique sans filtre tenant
- Variables d'environnement sensibles (DATABASE_URL, NEXTAUTH_SECRET) ne sont jamais committées
- Utiliser Prisma.JsonNull (pas null TypeScript) pour les champs Json nullable dans audit_logs

10. Glossaire

Terme	Définition
SaaS	Software as a Service — logiciel hébergé en ligne, accessible via navigateur sans installation
Multi-tenant	Architecture où une seule instance de l'application sert plusieurs clients isolés
Tenant	Un client (pharmacie) dans l'architecture multi-tenant
pharmacield	Identifiant unique d'une pharmacie, présent sur toutes les tables pour isoler les données
MVP	Minimum Viable Product — version minimale utilisable en production
CORE	Fonctionnalité indispensable au MVP — développée en priorité absolue
FIFO	First Expired First Out — lots dont la péremption est la plus proche consommés en premier
Lot	Ensemble d'unités d'un médicament livrées ensemble avec la même date de péremption
POS	Point of Sale — interface de caisse pour enregistrer les ventes
SSE	Server-Sent Events — technologie de mises à jour serveur → client en temps réel
CMV ⚡ v2.3	Coût des Marchandises Vendues — somme des prix d'achat FIFO des médicaments effectivement vendus
Bénéfice net ⚡ v2.3	CA net – CMV – Dépenses opérationnelles — premier indicateur de rentabilité réelle
Info trésorerie ⚡ v2.3	Total des achats reçus sur la période — distinct du bénéfice, reflète les décaissements
seuilAlerte ⚡ v2.3	Montant mensuel maximum par catégorie de dépense — alerte dashboard si dépassé
archivee ⚡ v2.3	Champ boolean sur dépenses — jamais de suppression physique en BDD
Paiement partiel ⚡ v2.4	Vente où le client ne paie qu'une partie — le reste est enregistré avec un motif dans credit_vente
credit_vente ⚡ v2.4	Table enregistrant les restes dus après paiement partiel : motif, montant, échéance, statut
Prise en charge ⚡ v2.4	Type de paiement partiel où le motif est une institution (Ministère, mutuelle, employeur...)
PlafondCredit ⚡ v2.4	Montant maximum de crédit accordé à un client — défini par l'Admin
soldeActuel ⚡ v2.4	Somme de tous les montants restants dus par un client — calculé dynamiquement
PWA	Progressive Web App — application web installable sur téléphone, fonctionne hors ligne

Terme	Définition
Service Worker	Programme JavaScript installé dans le navigateur qui intercepte les requêtes et gère le cache
IndexedDB	Base de données locale dans le navigateur — utilisée pour stocker les ventes hors ligne
Background Sync	API navigateur qui synchronise les données hors ligne au retour de la connexion
Snapshot prix	Prix enregistré au moment de la vente — immuable même si le prix change ensuite
Session caisse	Période de travail d'un caissier entre ouverture (fonds initial) et clôture (comptage)
Audit log	Enregistrement immuable d'une action sensible avec les données avant/après
JWT	JSON Web Token — format de token sécurisé utilisé pour les sessions NextAuth
ORM	Object-Relational Mapper — Prisma traduit les objets TypeScript en requêtes SQL
Super Admin	Compte PharmaGest (développeur) — accès à toutes les pharmacies pour gérer les licences
Licence unique	Modèle commercial : paiement one-time pour un accès permanent au logiciel
findFirst	Méthode Prisma à utiliser avec filtre pharmacield — remplace findUnique pour sécurité multi-tenant
Prisma.JsonNull	Valeur Prisma pour champs Json nullable — null TypeScript n'est pas assignable
Controller (RHF)	Composant react-hook-form à utiliser avec shadcn/ui — register ne propage pas le ref